

Corrigé - Examen Ingénierie Logicielle (HAI721) - Session 1 (2022-2023)

Durée : 2h00. Précision et concision notées. Codes en Java (comme l'énoncé), mais langage équivalent autorisé.

A Framework - Architectures extensibles (6 points)

1. L'architecture est le **pattern Composite** (composé).

- Classe abstraite `ComposantOrdi` (composant).
- Sous-classes feuilles : `ComposantSimple` (abstraite, optionnelle pour factorisation) avec sous-classes concrètes comme `RAM` (implémente `prixHT()` fixe).
- Sous-classe composite : `Montage` (contient `List<ComposantOrdi>`, somme les `prixHT()`).
- Sous-classes spécifiques : `CarteMereMontee` et `Ordinateur` étendent `Montage`.

Diagramme de classes UML (texte) :

```
ComposantOrdi (abstract)
- prixTTC() : double
+ getTVA() : double (protected)
+ prixHT() : double (abstract protected)
```

↑

```
ComposantSimple (abstract, optional)
```

↑

```
RAM
```

```
- prixBase : double
+ prixHT() : return prixBase
```

↑ (from ComposantOrdi)

```
Montage
```

```
- composants : List<ComposantOrdi>
+ add(ComposantOrdi c)
+ prixHT() : sum of composants.prixHT()
```

↑

```
CarteMereMontee
```

```
Ordinateur
```

2. `prixTTC()` est une fonction d'ordre supérieur car elle compose son comportement avec `prixHT()` (abstraite, fournie par sous-classes). La liaison dynamique (résolution runtime du type concret) permet le polymorphisme : l'appel à `prixHT()` invoque la version de la sous-classe, rendant `prixTTC()` extensible sans modification.

3.

```
public class Montage extends ComposantOrdi {
    private List<ComposantOrdi> composants = new ArrayList<>();

    public void add(ComposantOrdi c) {
        composants.add(c);
    }

    @Override
    protected double prixHT() {
        double somme = 0;
        for (ComposantOrdi c : composants) {
            somme += c.prixHT();
        }
        return somme;
    }
}
```

(Méthodes utiles : `add` et `prixHT` . Pas d'accesseurs comme demandé.)

4. Points d'extension (hot spots) : méthodes abstraites comme `prixHT()` (sous-classes définissent le calcul spécifique) ; création de nouvelles sous-classes de `ComposantOrdi` OU `Montage` .

Inversions de contrôle : le framework contrôle le flux (e.g., `prixTTC()` appelle `prixHT()` polymorphiquement).

Injection de dépendances : via `add(ComposantOrdi c)` dans `Montage` (dépendances injectées runtime, sans couplage fort).

5. Non, car `add` retourne `void` (pas de chaînage). Modification :

```
public Montage add(ComposantOrdi c) {
    composants.add(c);
    return this;
}
```

Permet le fluent interface : `new Ordinateur().add(...).add(...)` (fonctionne pour sous-classes de `Montage` comme `Ordinateur`).

B Amélioration de l'architecture (1) - Cohérence des montages (3 points)

1. Pattern **Fabrique Abstraite (Abstract Factory)** pour créer familles de montages préconfigurés et cohérents.

Architecture :

- Interface `FabriqueMontage` (abstraite) : `Montage createMontage(char config);`
- Classes concrètes : `FabriqueMontagePredefini1` implémente `FabriqueMontage` (crée `MontagePredefini1` avec config 'A' : ajoute composants compatibles, e.g., processeur adapté à carte-mère).
- `MontagePredefini1` étend `Montage` (constructeur privé ou protected ; config via fabrique).

Code clé :

```
public interface FabriqueMontage {
    Montage createMontage(char config);
}

public class FabriqueMontagePredefini1 implements FabriqueMontage {
    @Override
    public Montage createMontage(char config) {
        Montage m = new MontagePredefini1();
        if (config == 'A') {
            m.add(new CarteMere("Compatible"));
            m.add(new Processeur("Intel-i5")); // Cohérence assurée
            // Ajouts validés (lancer exception si incohérent)
        } else {
            throw new IllegalArgumentException("Config invalide");
        }
        return m;
    }
}
```

Utilisation : `FabriqueMontage fab = new FabriqueMontagePredefini1(); Montage m = fab.createMontage('A');`

Extensible : nouvelles fabriques pour autres montages prédefinis.

C Extensibilité et Typage Statique (6 points)

1.

a) Oui, les deux compilent séparément :

- `boolean equiv(Montage c, String critere)` : surcharge (overload, signature différente).
- `boolean equiv(ComposantOrdi c, String critere)` : redéfinition (override, même signature).

b) Choisir `boolean equiv(ComposantOrdi c, String critere)` (override) pour respecter le polymorphisme et Liskov : permet d'appeler via référence `ComposantOrdi` et résoudre dynamiquement ; évite casts et maintient extensibilité.

2. (En supposant override choisi : `equiv(ComposantOrdi c, String critere)` dans `Montage`.)

Les 5 envois (5 à 9) :

- 5 : `m2.equiv(m4,"x")` → `Montage.equiv` (receveur statique/dynamique `Montage`).
- 6 : `m3.equiv(m4,"x")` → `ComposantOrdi.equiv` (receveur statique/dynamique `ComposantOrdi` / `RAM`).
- 7 : `m4.equiv(m4,"x")` → `Montage.equiv` (receveur dynamique `Montage`, liaison dynamique).
- 8 : `m4.equiv((Montage)m4,"x")` → `Montage.equiv` (argument casté → surcharge non applicable ; override via dynamique).
- 9 : `m4.equiv(m3,"x")` → `Montage.equiv` (receveur dynamique `Montage`).

Explication : Typage statique détermine la surcharge (choix méthode par types arguments à compile-time). Liaison dynamique/polymorphisme d'inclusion résout la redéfinition (override) runtime via type dynamique du receveur.

3. `ComposantOrdi m4 = m2;` illustre le **principe Open/Closed** des frameworks : code client manipule via interface abstraite (`ComposantOrdi`), extensible par nouvelles sous-classes (`Montage`) sans modifier client ; polymorphisme assure comportement correct.

4. Solution : **double dispatch** (ou Visitor simplifié) pour éviter tests de type.

Ajouter méthode `boolean equivTo(ComposantOrdi other, String crit)` dans `ComposantOrdi` (abstraite ou default false).

Dans `equiv(ComposantOrdi c, String crit)`, appeler `c.equivTo(this, crit)` (inversion : argument "visite" receveur).

Sous-classes redéfinissent `equivTo` pour cas spécifiques (e.g., dans `RAM` : si `other instanceof RAM`, comparer specs ; else false).

Pour `m4.equiv(m1,"x")` : `Montage.equiv` appelle `m1.equivTo(m4, "x")` → exécute code de `RAM` sans test explicite dans `Montage`.

Extensible : nouvelles sous-classes ajoutent `equivTo` sans modifier existant.

D Amélioration de l'architecture (2) (5 points)

1. **State** : Gère états mutables (e.g., variation active/inactive), mais ici variations additives/multiples, pas exclusives ; moins adapté car état unique par objet, complexifie enchaînements.

Decorator : Ajoute comportements (variations) dynamiquement, en couches, sans modifier classe

originale ; idéal pour extensibilité modulaire (nouvelles variations sans toucher code existant).

Choix : **Decorator**, car variations cumulables (e.g., transport + matières), appliquées runtime, respecte Open/Closed.

2. Classes clés :

- `abstract class DecoratorPrix extends ComposantOrdi { protected ComposantOrdi wrapped; public DecoratorPrix(ComposantOrdi w) { wrapped = w; } }`
- Concrètes : `VariationTransport extends DecoratorPrix { @Override protected double prixHT() { return wrapped.prixHT() + (wrapped.getPoids() * facteurTransport); } }` (similaire pour `VariationMatiere`).

Envois clés :

- Création : `ComposantOrdi ramVariee = new VariationTransport(new VariationMatiere(new RAM()));` (chaînage extensible).
- Calcul : `ramVariee.prixTTC()` → appelle `prixHT()` du decorator outer → somme variations + `wrapped.prixHT()` (récursif jusqu'à feuille).

Code `prixHT()` (dans decorators) :

```
@Override
protected double prixHT() {
    return wrapped.prixHT() + calculVariation(); // abstract calculVariation() pour sous-classes
}
```

Extensible : nouvelles variations (e.g., `VariationDouane`) étendent `DecoratorPrix` , appliquées sans modifier `RAM` ou framework.